

Health Insurance Analytics Platform

Production-Grade Data Engineering + ML Pipeline, Deployed on AWS

A 20-day, day-by-day execution plan — from raw policy/claims data to a monitored, CI/CD-deployed prediction API on AWS.

Prepared for: Affan · Final-year CSE · Data Engineering track

1. Project Overview & Objectives

You have a multi-year (3 periods), policy/insured-level health insurance dataset covering policy lifecycle dates, lapse status, premium, claims cost, service utilisation, and geographic/socioeconomic concentration indices. The goal is to turn this into a **production-style analytics platform**, not a single notebook: a repeatable data pipeline, two supporting ML models, a served prediction API, and the AWS infrastructure and CI/CD needed to run it like a real product. This is scoped to be finished, deployed, and demoable in 20 working days, and is designed to double as a strong data-engineering portfolio piece and interview talking point.

1.1 What you are building

- **Lapse / churn model** — predicts the *lapse* outcome (1: early lapse, 2: active, 3: lapse at expiration) for an insured-year record, so the business can flag at-risk policies before renewal.
- **Claims cost model** — regresses *cost_claims_year* to estimate expected claims cost per insured for the coming period, useful for pricing and reserving.
- **A data lake + ETL pipeline** on S3/Glue that turns raw yearly extracts into clean, versioned, queryable feature tables — the same shape a real insurer's data platform would need.
- **A served inference API** (FastAPI + SageMaker) behind API Gateway, deployed via Docker/ECS, with CI/CD, infrastructure-as-code, and monitoring — the part most portfolio projects skip.

1.2 Success criteria

- End-to-end pipeline runs from raw S3 upload to a live prediction returned by an HTTPS endpoint.
- Both models beat sensible baselines (majority-class / mean-prediction) with tracked metrics in MLflow.
- All infrastructure is defined in Terraform and rebuildable from scratch (`terraform apply`) — nothing clicked manually in the AWS console.
- A GitHub push triggers tests, a Docker build, and an automatic redeploy.
- Basic monitoring (latency, error rate, data drift) is visible on a CloudWatch dashboard.

2. High-Level Architecture

The platform follows a standard medallion data-lake pattern feeding a SageMaker-based MLOps loop, served through a containerised API. Every box below maps to a Terraform module — see Section 6.

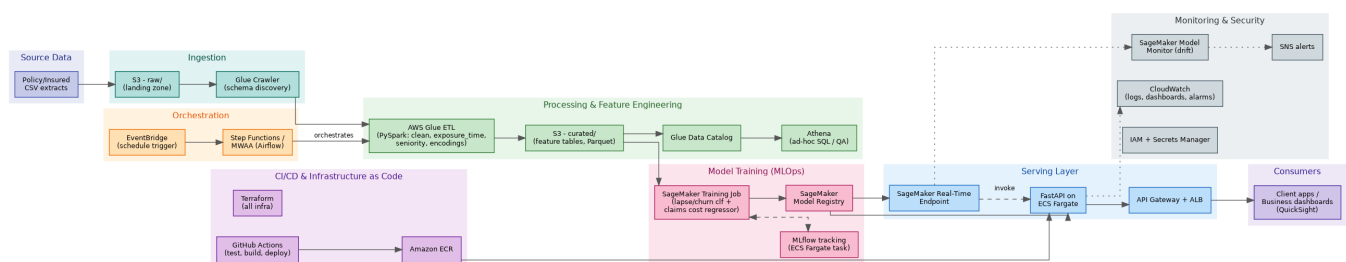


Figure 1 — End-to-end architecture: ingestion → data lake → training → registry → serving → monitoring.

2.1 Data flow, in words

- **Ingest:** yearly CSV extracts land in **S3 raw**; a Glue Crawler infers/updates schema.
- **Orchestrate:** an EventBridge schedule (or manual trigger) kicks off a Step Functions state machine that runs the Glue ETL job and, on success, the SageMaker training pipeline.

- **Transform:** a PySpark Glue job cleans dates, recomputes/validates *exposure_time*, *seniority_insured*, *seniority_policy*, encodes categoricals, and writes Parquet to **S3 curated/**, cataloged for Athena.
- **Train:** a SageMaker Pipeline reads curated features, trains the lapse classifier and claims-cost regressor, logs runs to MLflow, and registers approved models in the SageMaker Model Registry.
- **Serve:** the approved model is deployed to a SageMaker real-time endpoint; a FastAPI service on ECS Fargate wraps it with business logic/validation and is exposed via API Gateway + ALB.
- **Monitor:** CloudWatch captures logs/metrics, SageMaker Model Monitor watches for feature drift, and SNS emails you on alarms.

3. Tech Stack

Layer	Tooling
Languages	Python 3.11, SQL
Data processing	PySpark (AWS Glue), pandas, PyArrow
Modeling	scikit-learn, XGBoost / LightGBM, SHAP, Optuna
Experiment tracking	MLflow (tracking server on ECS Fargate + S3 artifact store)
Orchestration	AWS Step Functions (EventBridge-triggered); MWAA/Airflow as a stretch goal
Storage & catalog	Amazon S3 (data lake), AWS Glue Data Catalog
Query / QA	Amazon Athena
Training & registry	Amazon SageMaker (Pipelines, Training Jobs, Model Registry)
Serving	SageMaker real-time endpoint, FastAPI + Uvicorn, Docker, ECS Fargate
Networking	API Gateway (HTTP API), Application Load Balancer, VPC with private subnets
IaC	Terraform (modules per service, remote state in S3 + DynamoDB lock table)
CI/CD	GitHub Actions (lint, test, build, push to ECR, deploy)
Monitoring	CloudWatch (logs, dashboards, alarms), SageMaker Model Monitor, SNS
Security	IAM least-privilege roles, Secrets Manager, S3 bucket policies + encryption (SSE-S3/KMS)

4. Repository Structure

```

insurance-analytics-platform/
■■■ infra/                                # Terraform modules
■   ■■■ s3.tf glue.tf sagemaker.tf ecs.tf api_gateway.tf
■   ■■■ iam.tf networking.tf monitoring.tf variables.tf
■■■ etl/
■   ■■■ glue_jobs/clean_and_feature_engineer.py
■   ■■■ glue_jobs/validate_schema.py
■■■ ml/
■   ■■■ training/train_lapse_model.py
■   ■■■ training/train_claims_cost_model.py
■   ■■■ pipelines/sagemaker_pipeline.py
■   ■■■ evaluation/evaluate.py
■■■ serving/
■   ■■■ app/main.py                      # FastAPI app
■   ■■■ app/schemas.py                  # Pydantic request/response models
■   ■■■ Dockerfile
■   ■■■ tests/
■■■ notebooks/
■   ■■■ 01_eda.ipynb
■■■ .github/workflows/
■   ■■■ ci.yml                          # lint + test on every PR
■   ■■■ deploy.yml                      # build, push, terraform apply, smoke test
■■■ data_dictionary.md                  # from the variable table you already have
■■■ requirements.txt
■■■ README.md

```

5. 20-Day Execution Plan

Each day assumes a focused ~4-6 hour block. Phases deliberately front-load the data engineering work (days 1-4) since that's both the hardest part to get right and the part most valuable for a DE portfolio. If a day overruns, prefer to cut scope (e.g. skip MWAA, keep EventBridge+Step Functions) rather than skip testing or IaC.

Day	Phase	Key Tasks	Deliverable
1	Setup	Init git repo + folder structure; create AWS account/IAM user with scoped permissions; Python venv + requirements.txt; create S3 buckets (raw/curated/artifacts) via console once, then import into Terraform later; upload the 3 yearly extracts to S3 raw/.	Repo skeleton + data staged in S3
2	EDA	Profile schema, null rates, and value ranges for every field in your data dictionary; plot distributions of lapse, premium, cost_claims_year, exposure_time; check ID continuity across the 3 periods; log data-quality issues (bad dates, exposure_time outside [0,1], etc.).	EDA notebook + data quality notes
3	Data engineering I	Write the Glue PySpark job: type casting, date parsing, null handling, de-duplication per (ID, period); recompute/validate exposure_time and seniority_* fields against raw dates as a sanity check.	Glue ETL job v1 → curated bronze table
4	Data engineering II	Feature engineering: tenure buckets, encode type_policy / type_policy_dg / type_product / distribution_channel / gender / C_H / C_C, derive claim_frequency = n_medical_services/exposure_time and loss_ratio = cost_claims_year/premium, bucket the concentration indices (IICIMUN, IICIPROV, C_GI, C_II).	Curated feature table + Glue Catalog + Athena validation queries
5	Modeling setup	Define the two targets precisely: lapse classification (target = lapse, temporal split — train on periods 1-2, validate on period 3) and claims cost regression (target = cost_claims_year, evaluated on records with exposure_time > 0). Build baseline models.	Baseline metrics logged in MLflow
6	Modeling iteration	Train XGBoost/LightGBM for both targets; hyperparameter search with Optuna; handle class imbalance in lapse (esp. code 1) with class weights or resampling.	Tuned candidate models + MLflow runs
7	Model evaluation	Evaluate lapse model (AUC, F1, confusion matrix per class) and claims-cost model (RMSE, MAE, Gini/lift chart); run SHAP for explainability; pick champion models.	Evaluation report + champion models selected
8	SageMaker packaging I	Containerize training code; port training scripts to SageMaker Training Job entry points (train() + model_fn/input_fn for inference).	Docker training image runs locally
9	SageMaker packaging II	Assemble a SageMaker Pipeline: Processing step (feature prep) → Training step → Evaluation step → Conditional Register step into the Model Registry.	SageMaker Pipeline executes end-to-end on a subset
10	Serving I	Build the FastAPI app: /predict/lapse and /predict/claims_cost endpoints, Pydantic request/response schemas matching your feature set, model loading from S3/registry.	FastAPI app runs locally, /docs works
11	Serving II	Write pytest unit + integration tests for the API; Dockerize the service; test with docker-compose against a local model artifact.	Tested, containerized inference service
12	Serving III	Deploy a SageMaker real-time endpoint for the champion models; wire the FastAPI service to call it (or embed the model directly if simpler); manual test with sample payloads.	Live endpoint returns predictions
13	IaC I	Terraform: S3 buckets, IAM roles/policies (least privilege per service), ECR repo, remote state backend (S3 + DynamoDB lock).	terraform apply creates core storage/identity resources

Day	Phase	Key Tasks	Deliverable
14	IaC II	Terraform: Glue jobs/crawlers, EventBridge rule, Step Functions state machine, SageMaker resources (pipeline, endpoint config).	Data + ML infra reproducible from code
15	IaC III	Terraform: VPC/subnets, ECS Fargate service + task definition, ALB, API Gateway, CloudWatch log groups, SNS topic.	Full stack deployable with one command
16	CI/CD I	GitHub Actions CI workflow: lint (black/flake8), run pytest, build Docker image on every PR.	Green CI pipeline on PRs
17	CI/CD II	GitHub Actions CD workflow: on merge to main, push image to ECR, run terraform apply (or targeted update), update ECS service, run a smoke test against the live API, rollback on failure.	Push-to-deploy pipeline working
18	Monitoring	Build a CloudWatch dashboard (latency, 4xx/5xx rate, invocation count); enable SageMaker Model Monitor for drift on key inputs (age, premium, exposure_time); wire SNS email alerts.	Live dashboard + at least one working alarm
19	Docs & polish	Write README with the architecture diagram, setup steps, and API docs; write the data dictionary file from your variable table; write short ADRs for major decisions (e.g. why Glue over raw Spark-on-EMR).	Documentation complete
20	Final test & demo	Run the full flow raw-data-to-prediction end to end; prepare a 5-minute demo script and talking points; tag v1.0; scale down/stop non-essential resources (endpoints, Fargate tasks) to control cost.	Working, demoable v1.0 + cost cleaned up

6. AWS Services — Purpose & Notes

Service	Purpose in this project
S3	Data lake (raw/curated/artifacts), Terraform remote state, model artifact storage. Enable versioning + SSE encryption; separate buckets or clearly prefixed folders per zone.
Glue (Jobs + Crawler + Catalog)	Serverless Spark ETL and schema catalog — avoids running/managing EMR clusters for a dataset this size.
Athena	Ad-hoc SQL over the Glue Catalog for QA and quick aggregate checks without a warehouse.
Step Functions + EventBridge	Orchestrates ETL → training on a schedule; simpler and cheaper than MWAA for a project this scale (mention MWAA as a scale-up option in interviews).
SageMaker (Pipelines, Training, Registry, Endpoints)	Managed training + versioned model registry + hosted real-time inference, with built-in autoscaling and Model Monitor for drift.
ECS Fargate	Runs the FastAPI wrapper and the MLflow tracking server without managing EC2 instances.
ECR	Stores Docker images built by CI for the FastAPI service (and training container if custom).
API Gateway + ALB	Public HTTPS entry point in front of the FastAPI service; API Gateway for throttling/keys, ALB for routing to Fargate tasks.
IAM	One scoped role per service (Glue, SageMaker, ECS task, GitHub OIDC deploy role) — never a shared admin role.
Secrets Manager	Any credentials the pipeline needs (MLflow DB creds, third-party keys).
CloudWatch	Centralized logs, custom metrics/dashboards, alarms feeding SNS.
SNS	Alert delivery (email/SMS) for alarms and drift notifications.

7. Evaluation Metrics & Baselines

- **Lapse classifier:** macro-F1 and per-class recall (class 1 — early lapse — is the business-critical, usually-minority class); ROC-AUC vs. a majority-class baseline.
- **Claims-cost regressor:** RMSE and MAE in euros, plus a Gini/lift chart vs. a mean-prediction baseline — standard in actuarial pricing work, and a strong thing to mention in interviews.
- **System:** p95 API latency, error rate, and successful end-to-end pipeline runs — these are the "production" metrics that separate this from a notebook project.

8. Cost Control

- Use SageMaker on-demand endpoints only while actively testing/demoing; delete or use serverless inference between sessions.
- Keep Glue job runs small (this dataset is modest — a small DPU count is enough); avoid always-on clusters.
- Use Fargate Spot for the MLflow tracking server (non-critical, restartable).
- Set a AWS Budget alert (e.g. \$20-30) on day 1 so nothing runs away unnoticed.
- Tear down (terraform destroy on non-persistent modules) between work sessions if cost-sensitive; keep S3 and Terraform state.

9. Risks & Mitigations

Risk	Mitigation
Temporal leakage between train/validation	Split strictly by period (year), never randomly, given the same insured can appear across years.
Severe class imbalance on lapse=1	Use class weights / focal-loss-style objectives; report per-class recall, not just accuracy.
Scope creep on infra (MWAA, multi-account, blue/green)	Ship the simple version first (EventBridge + Step Functions, single AWS account); note advanced options as "future work" in the README.
AWS costs spiraling	Budget alert on day 1; destroy non-persistent infra between sessions (see Section 8).
Running out of time on CI/CD/IaC (days 13-17)	These are compressible if needed — a working terraform apply for the core path (S3, SageMaker endpoint, ECS) matters more than every module being polished.

10. Interview / Portfolio Talking Points

- "I built a medallion-architecture data lake on S3/Glue and a temporally-correct train/validation split for a longitudinal insurance dataset."
- "I shipped two models (classification + regression) through a SageMaker Pipeline into a Model Registry, not just a notebook."
- "The whole stack — data pipeline, training, serving, monitoring — is defined in Terraform and deployed via GitHub Actions; nothing was clicked in the console."
- "I instrumented drift detection and CloudWatch dashboards so the system tells you when it's wrong, not just when it's down."

Note: this plan assumes ~4-6 focused hours/day. If you're doing this alongside coursework, treat the 20 days as 20 work-sessions rather than 20 calendar days — the sequencing (data → models → serving → infra → CI/CD → monitoring) matters more than the exact daily boundaries.